

北京邮电大学 • 计算机网络实验

协议数据的捕获和解析

实验二 • 实验报告

姓 名 张恒基
学 号 2024210926
班 级 2024211301
完成日期 2026 年 6 月

北京邮电大学

Beijing University of Posts and Telecommunications

摘要

本报告围绕北京邮电大学计算机网络实验二「协议数据的捕获和解析」，在 Windows 11 校园网环境下使用 Wireshark 对 IP、ICMP、DHCP、ARP、TCP 五类典型协议进行抓包与字段级分析。实验依次完成 ICMP ping（含大包分片）、DHCP 地址申请（DORA）、ARP 地址解析以及 TCP 连接建立、数据传输与释放全过程的捕获；在 Wireshark 中展开协议树，抄录首部字段，验证 IPv4 首部校验和算法，分析 IP 分片标识与偏移关系，对比 ICMP Echo 请求/应答、ARP 广播/单播差异，并追踪 TCP 三次握手、确认应答与四次挥手中的序号规律。

全文按指导书第 9 节结构组织，所有字段值均从 `captures/` 下 `.pcapng` 经 tshark 解析抄录，并附 Wireshark 三栏截图与 Mermaid 时序图；同时完成了校验脚本编写、截图自动化与 Typst 报告工程化整理。

关键词：Wireshark • IP 分片 • ICMP • DHCP • ARP • TCP 握手

目录

摘要	2
1 实验目的	1
2 实验环境	2
3 实验内容和实验步骤	3
3.1 实验任务	3
3.2 ICMP 数据捕获	3
3.3 DHCP 数据捕获	3
3.4 ARP 数据捕获	3
3.5 TCP 数据捕获	4
3.6 协议分析方法	4
4 IP 协议分析	5
4.1 IP 首部字段	5
4.2 IP 首部校验和验证	6
4.3 IP 分片分析	7
5 ICMP 协议分析	9
6 DHCP 协议分析	11
6.1 DHCP 报文过程 (DORA)	11
6.2 DHCP 字段和配置参数	12
7 ARP 协议分析	14
8 TCP 协议分析	16
8.1 TCP 首部字段	16
8.2 三次握手	16
8.3 数据传输	17
8.4 四次挥手	18
9 实验实现与工作量说明	20
9.1 代码与文件规模说明	21

10 实验中遇到的问题和解决方案 23

11 实验心得 24

12 可复现性说明 25

13 附录：工程目录结构 26

实验目的

通过本实验，我希望达成以下目标：

1. 掌握网络协议分析仪的使用方法：熟悉 Wireshark 的捕获过滤器、显示过滤器，以及 Packet List / Packet Detail / Packet Bytes 三栏的协同分析方式。
2. 理解 IP 层数据报结构：能够读懂 IPv4 首部各字段含义，手工验证首部校验和，并解释大包 ping 触发的 IP 分片机制。
3. 认识常见网络控制协议：分析 ICMP 诊断报文、DHCP 动态配置流程、ARP 地址解析过程，建立「协议字段 ↔ 网络行为」的对应关系。
4. 深入理解 TCP 可靠传输：从真实抓包中观察三次握手、序号确认、窗口与 MSS，以及连接释放的四次挥手过程。

实验环境

操作系统	Windows 11
抓包工具	Wireshark 4.6.6 + Npcap
网络环境	校园 Wi-Fi (WLAN)
本机 IP	10.122.219.108
子网掩码	255.255.192.0
默认网关	10.122.192.1
本机 MAC	c0:35:32:78:d3:09 (以 ipconfig /all 为准)

表 1 实验软硬件环境

实验内容和实验步骤

实验任务

本实验需完成以下五项协议分析任务：

1. IP 协议：首部字段表、首部校验和验证、分片分析 ICMP 协议：Echo Request / Reply 字段对比 DHCP 协议：DORA 四阶段与配置参数提取 ARP 协议：Request / Reply 字段与广播/单播分析 TCP 协议：首部字段、三次握手、数据传输、四次挥手

ICMP 数据捕获

过滤器与命令

- Capture Filter: icmp
- Display Filter: icmp
- 普通 ping: ping www.bupt.edu.cn
- 大包分片: ping -l 8000 10.122.192.1

操作步骤：在 WLAN 网卡开始捕获 → 执行 ping 命令 → 停止捕获 → 分别保存为 captures/icmp/icmp-normal.pcapng 与 captures/icmp/icmp-large-packet-fragmentation.pcapng。

DHCP 数据捕获

过滤器与命令

- Capture Filter: udp port 67
- Display Filter: bootp
- 释放 IP: ipconfig /release
- 重新申请: ipconfig /renew （需管理员权限）

保存为 captures/dhcp/dhcp-dora.pcapng，验证 Transaction ID 相同的 Discover / Offer / Request / ACK 四包齐全。

ARP 数据捕获

过滤器与命令

- Capture Filter: `arp`
- Display Filter: `arp`
- 清空缓存: `arp -d *`
- 触发解析: `ping 10.122.192.1`

保存为 `captures/arp/arp-request-reply.pcapng`。

TCP 数据捕获

过滤器与命令

- Capture Filter: `tcp port 80`
- Display Filter: `tcp.port == 80`
- 触发流量: `curl -4 http://example.com`

保存为 `captures/tcp/tcp-http.pcapng`，确认含 SYN 握手、数据段与 FIN 挥手（`tcp.stream==0`）。

协议分析方法

本实验采用「过滤定位—会话归并—字段展开—机制验证」的分析方法。首先使用 Capture Filter 降低无关流量干扰，再使用 Display Filter 精确定位目标协议；随后根据 Transaction ID、Identification、五元组或 Stream Index 将属于同一次交互的报文归并到同一分析对象中；最后在 Packet Detail 中逐层展开 Ethernet、IP、ICMP、DHCP、ARP、TCP 等协议字段，并结合 Packet Bytes 中的十六进制原始数据验证字段含义。对于 IP 校验和、IP 分片偏移、TCP 序号确认关系等关键机制，不仅记录 Wireshark 解析结果，还通过手工计算或脚本进行交叉验证，避免只停留在截图描述层面。

分析时没有仅依据报文出现顺序判断归属，而是使用 Identification、Transaction ID、Identifier/Sequence Number、TCP Stream 等协议内字段对报文进行归并，保证分析对象属于同一次通信过程。常用 Display Filter 包括：`ip.id == 0xcfb`、`icmp.type == 8`、`icmp.type == 0`、`bootp.id == 0x348015f3`、`arp.opcode == 1`、`arp.opcode == 2`、`tcp.stream == 0`、`tcp.flags.syn == 1`、`tcp.analysis.retransmission`、`tcp.analysis.duplicate_ack` 等。

IP 协议分析

抓包文件：`captures/icmp/icmp-normal.pcapng`（帧 1）、`captures/icmp/icmp-large-packet-fragmentation.pcapng`（分片 Identification `0xCFBA`）。

IP 首部字段

选定帧 1：本机 `10.122.219.108` → `10.3.19.2`（ping `www.bupt.edu.cn`）的 ICMP Echo Request 承载 IPv4 数据报。

字段	捕获值	长度	功能说明
Version	4	4 bit	IPv4
Header Length	20 bytes (5)	4 bit	首部长度，单位 4 字节
Differentiated Services	0x00	8 bit	DSCP/ECN 均为 0
Total Length	60	16 bit	IP 数据报总长度（字节）
Identification	0xCFB5	16 bit	分片标识
Flags	DF=0, MF=0	3 bit	未分片
Fragment Offset	0	13 bit	分片偏移（×8 字节）
Time to Live	128	8 bit	生存时间 TTL
Protocol	ICMP (1)	8 bit	上层协议类型
Header Checksum	0x6820	16 bit	首部校验和
Source Address	10.122.219.108	32 bit	源 IP
Destination Address	10.3.19.2	32 bit	目的 IP

表 2 IPv4 首部字段（帧 1）

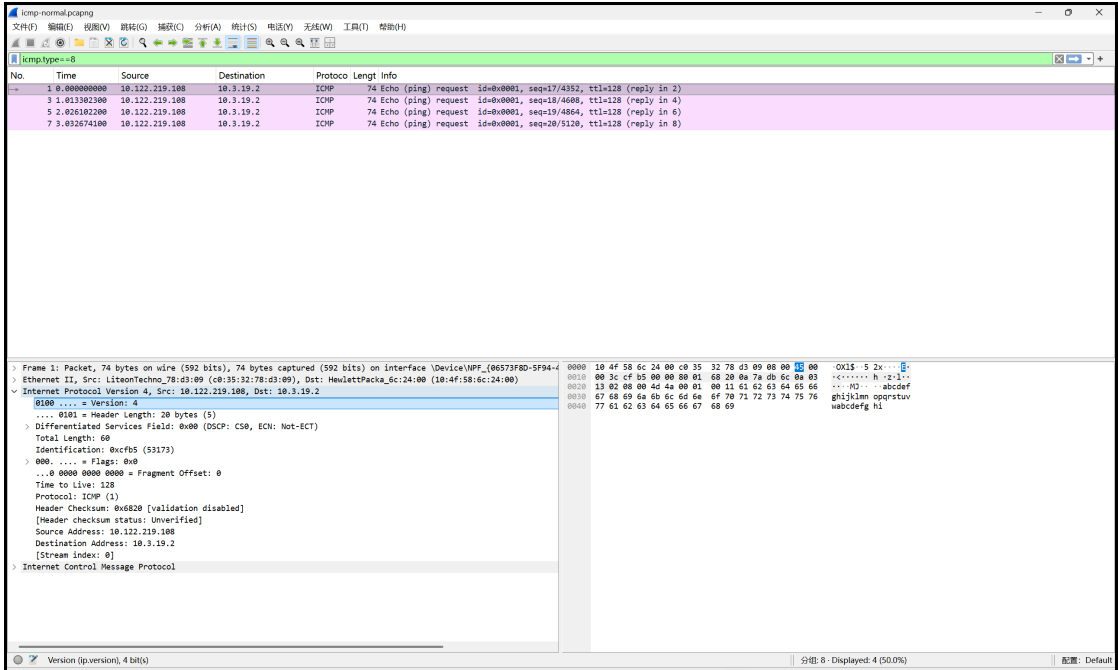


图 1 IPv4 首部字段 Wireshark 截图（见图）

Packet Bytes 首部十六进制（20 字节）：

```
45 00 00 3c cf b5 00 00 80 01 68 20 0a 7a db 6c 0a 03 13 02
```

IP 首部校验和验证

IPv4 首部校验和采用 16 位反码求和（one’s complement sum）：

1. 将校验和字段（第 11 - 12 字节）置 0；
2. 将首部按 16 bit 分组求和，溢出高位回卷到低位；
3. 对结果按位取反，即为校验和。

验证过程（帧 1）

1. 复制 20 字节首部：4500 003c cfb5 0000 8001 6820 0a7a db6c 0a03 1302
2. 校验和置零：4500 003c cfb5 0000 8001 0000 0a7a db6c 0a03 1302
3. 按 16 bit 分成 10 组：

组号	十六进制	十进制
1	0x4500	17664
2	0x003C	60
3	0xCFB5	53045
4	0x0000	0
5	0x8001	32769
6	0x0000	0

组号	十六进制	十进制
7	0x0A7A	2682
8	0xDB6C	56172
9	0x0A03	2563
10	0x1302	4866

表 3 IP 首部 16 bit 分组

4. 反码相加（含回卷）：

```
0x4500 + 0x003C = 0x453C
0x453C + 0xCFB5 = 0x114F1 → 回卷 → 0x14F2
0x14F2 + 0x8001 = 0x94F3
0x94F3 + 0x0A7A = 0x9F6D
0x9F6D + 0xDB6C = 0x17AD9 → 回卷 → 0x7ADA
0x7ADA + 0x0A03 = 0x84DD
0x84DD + 0x1302 = 0x97DF
```

5. 取反：`~0x97DF = 0x6820`

6. 与 Wireshark 对比：显示 `0x6820`，一致 ✓

`scripts/verify_ipv4_checksum.py` 交叉验证输出同为 `0x6820`。

IP 分片分析

`ping -l 8000` 抓包中 Identification = `0xCFBA` 的 6 个出站分片（帧 13 - 18）：

分片序号	Identification	MF	Fragment Offset	Total Length
1	0xCFBA	1	0	1500
2	0xCFBA	1	185	1500
3	0xCFBA	1	370	1500
4	0xCFBA	1	555	1500
5	0xCFBA	1	740	1500
6	0xCFBA	0	925	628

表 4 同一原始数据报的分片对比

分析说明：

- Identification 均为 0xCFBA，表明 6 个分片属于同一原始 IP 数据报。
- 前 5 个分片 MF=1，最后一个 MF=0，表示分片结束。

- Fragment Offset 递增: $0 \rightarrow 185 \rightarrow 370 \rightarrow \dots \rightarrow 925$ (单位 8 字节), 接收端按偏移重组。
- 除末片 628 字节外, 其余各片 1500 字节 (接近以太网 MTU)。

原始 IP 数据报 (> MTU)

分片1	分片2	分片3	分片4	分片5	分片6
off=0	off=185	off=370	off=555	off=740	off=925
MF=1	MF=1	MF=1	MF=1	MF=1	MF=0
len=1500	len=1500	len=1500	len=1500	len=1500	len=628

Identification = 0xCFBA (全部相同)

图 1 IP 分片示意图

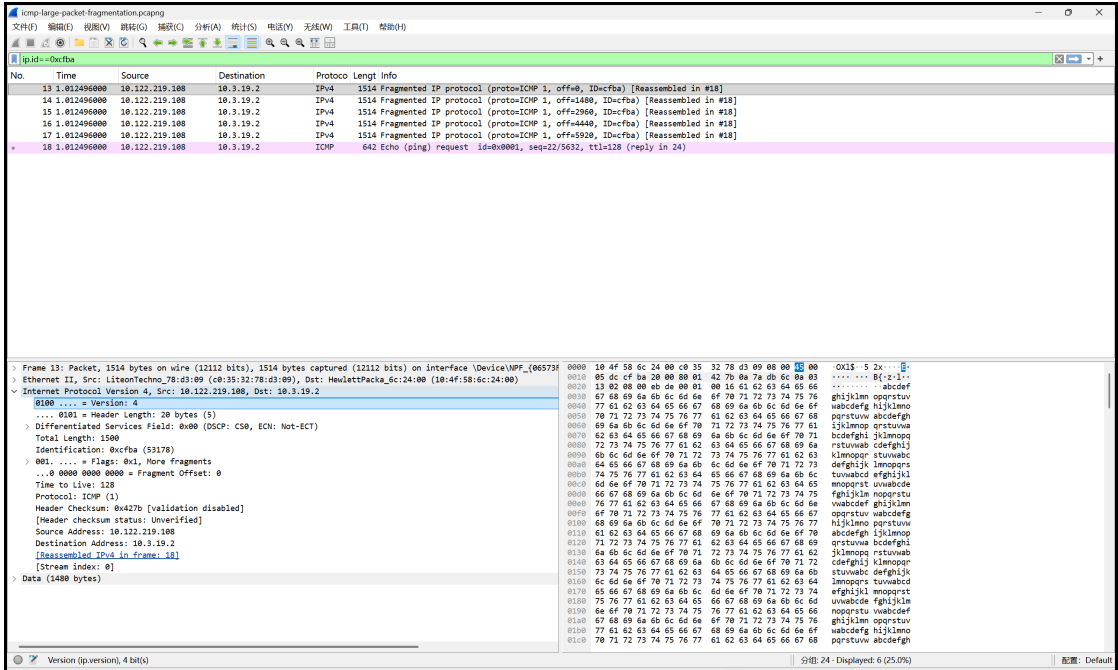


图 2 IP 分片对比 Wireshark 截图 (见图)

本节结论: 通过 Identification、MF 与 Fragment Offset 三个字段的配合, 可以验证 IP 层在 MTU 限制下的分片与重组机制; 同一原始数据报的所有分片共享相同 Identification, 接收端按偏移拼接后还原上层 ICMP 载荷。

ICMP 协议分析

抓包文件：`captures/icmp/icmp-normal.pcapng`（`ping www.bupt.edu.cn`）。

ICMP（Internet Control Message Protocol）是网络层控制报文协议，用于差错报告与网络诊断。`ping` 使用 Echo Request（Type=8）与 Echo Reply（Type=0）检测连通性。

选定帧 1（Request）与帧 2（Reply），序号均为 17：

字段	Request	Reply	功能
Type	8	0	报文类型
Code	0	0	类型细分
Checksum	0x4D4A	0x554A	ICMP 校验和
Identifier	1	1	ping 会话标识
Sequence Number	17	17	序号
Data	abcdefghijklmnopqrstuvwabcdefghi	abcdefghijklmnopqrstuvwabcdefghi	32 字节测试数据

表 5 ICMP Echo 请求与应答字段对比（帧 1 & 2）

会话关联说明：

- Identifier 相同（均为 1），表明属于同一次 `ping` 进程。
- Sequence Number 对应（均为 17），表明为第 17 号请求的应答。
- 源/目的 IP 互换：Request `10.122.219.108` → `10.3.19.2`，Reply `10.3.19.2` → `10.122.219.108`。
- Reply 的 IP TTL 为 58，Request TTL 为 128（Windows 默认）。

结论：Identifier + Sequence Number 双字段匹配，可唯一确定 Request/Reply 配对关系。

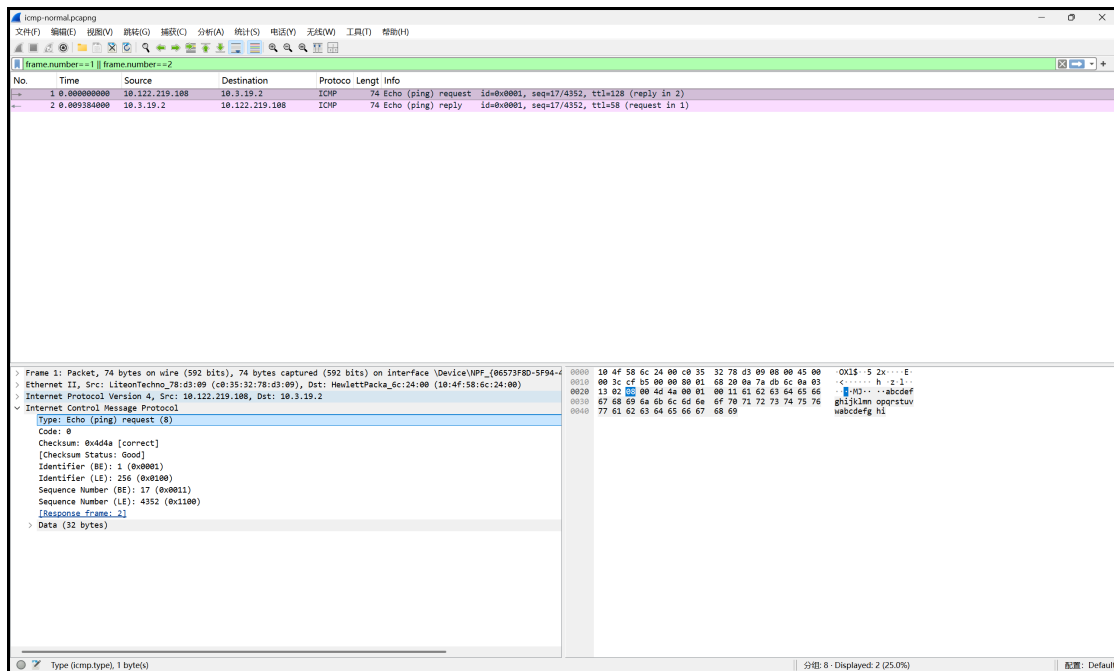


图 3 ICMP Echo Request / Reply Wireshark 截图（见图）

本节结论：ICMP Echo 通过 Type 8/0 区分请求与应答，Identifier 与 Sequence Number 共同标识同一次 ping 交互；结合源/目的 IP 互换，可在抓包中唯一配对 Request 与 Reply，验证网络层连通性诊断机制。

DHCP 协议分析

抓包文件：`captures/dhcp/dhcp-dora.pcapng`（`ipconfig /release` + `ipconfig /renew`）。
DHCP（Dynamic Host Configuration Protocol）动态主机配置协议，客户端无需手工配置即可从服务器获取 IP 地址、子网掩码、默认网关、DNS 服务器及租约时间等参数。

DHCP 报文过程（DORA）

Transaction ID 均为 `0x348015F3` 的四个包（帧 2 - 5）：

阶段	报文类型	源地址	目的地址	主要作用
1	DHCP Discover	0.0.0.0	255.255.255.255	客户端广播寻找服务器
2	DHCP Offer	10.3.9.2	10.122.219.108	服务器单播提供 IP
3	DHCP Request	0.0.0.0	255.255.255.255	客户端广播请求使用
4	DHCP ACK	10.3.9.2	10.122.219.108	服务器单播确认分配

表 6 DHCP DORA 四阶段

帧 1 为 DHCP Release (Transaction ID `0x9F16ECB3`)，属于 `/release` 操作，不属于本次 DORA 流程。

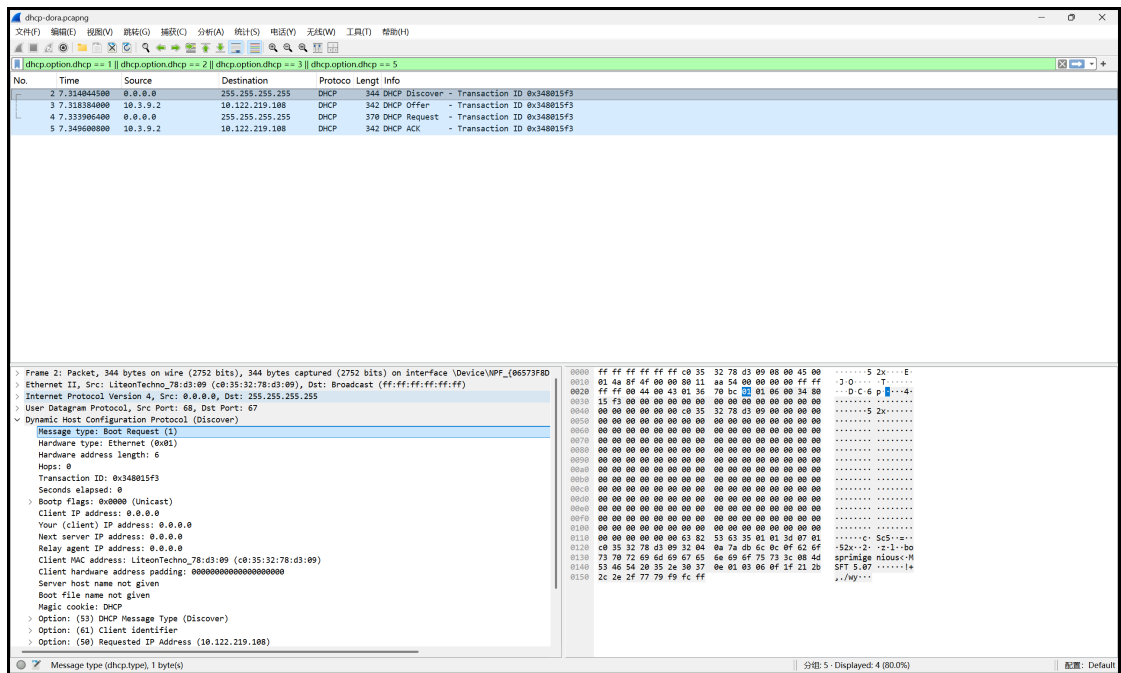


图 4 DHCP DORA 四包 Wireshark 截图（见图）

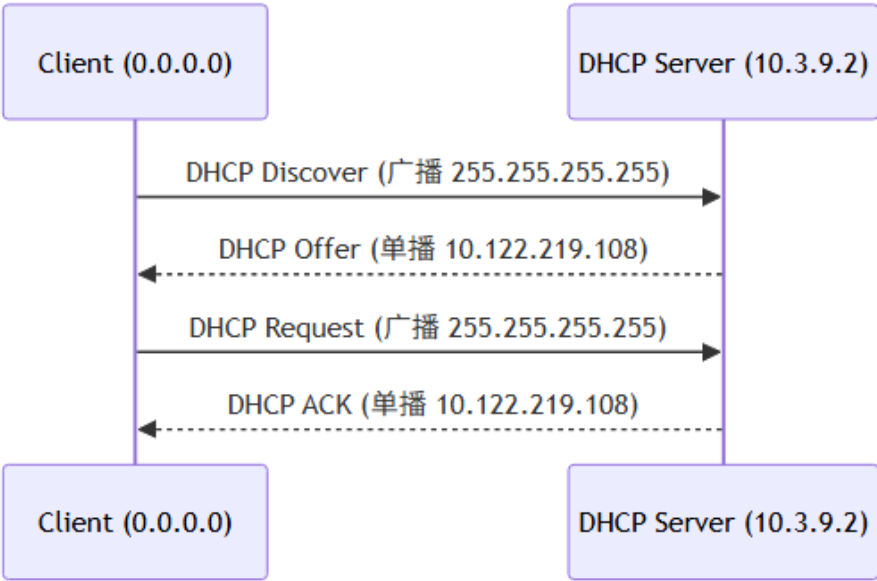


图 5 DHCP DORA 消息序列图

DHCP 字段和配置参数

从 DHCP ACK（帧 5）提取：

参数	Option	捕获值	含义
Your IP Address	—	10.122.219.108	分配给客户端的 IP
Subnet Mask	1	255.255.192.0	子网掩码
Router	3	10.122.192.1	默认网关

参数	Option	捕获值	含义
Domain Name Server	6	10.3.9.5, 10.3.9.4, 10.3.9.6	DNS 服务器
IP Address Lease Time	51	3600 (1 小时)	租约时间
DHCP Server Identifier	54	10.3.9.2	DHCP 服务器标识

表 7 DHCP ACK 配置参数

服务器角色判断：

- DHCP 服务器为 **10.3.9.2** (Option 54)，不是默认网关 **10.122.192.1**。
- 网关 **10.122.192.1** 仅作为 Option 3 (Router) 下发。
- giaddr 均为 0.0.0.0，客户端与服务器在同一广播域，无 DHCP Relay。

本节结论：DORA 四步完整展示了客户端从「无 IP (0.0.0.0)」到获得地址、掩码、网关、DNS 与租约的全过程；Discover/Request 采用广播、Offer/ACK 采用单播，体现了 DHCP 在局域网内的地址分配机制。

ARP 协议分析

抓包文件：`captures/arp/arp-request-reply.pcapng`（`ping 10.122.192.1` 触发）。

ARP（Address Resolution Protocol）根据 IP 地址解析 MAC 地址，使以太网帧能正确封装目的物理地址。

字段	Request	Reply	功能
Hardware Type	1	1	Ethernet
Protocol Type	0x0800	0x0800	IPv4
Opcode	1	2	请求 / 应答

表 8 ARP 固定字段（帧 1 & 2）

报文	Sender MAC	Sender IP	Target MAC	Target IP
Request	10:4f:58:6c:24:00	10.122.192.1	00:00:00:00:00:00	10.122.219.108
Reply	c0:35:32:78:d3:09	10.122.219.108	10:4f:58:6c:24:00	10.122.192.1

表 9 ARP 地址字段（帧 1 & 2）

广播与单播：

- ARP Request(帧 1)：Opcode=1, Target MAC 为 `00:00:00:00:00:00`。网关 `10.122.192.1` 向本机发起询问，以太网目的地址为本机 MAC `c0:35:32:78:d3:09`（单播 ARP）。
- ARP Reply（帧 2）：Opcode=2，本机以 `c0:35:32:78:d3:09` 单播回复网关 MAC `10:4f:58:6c:24:00`。

核心原理：用广播（或单播）询问谁拥有某 IP，目标主机单播返回自己的 MAC。

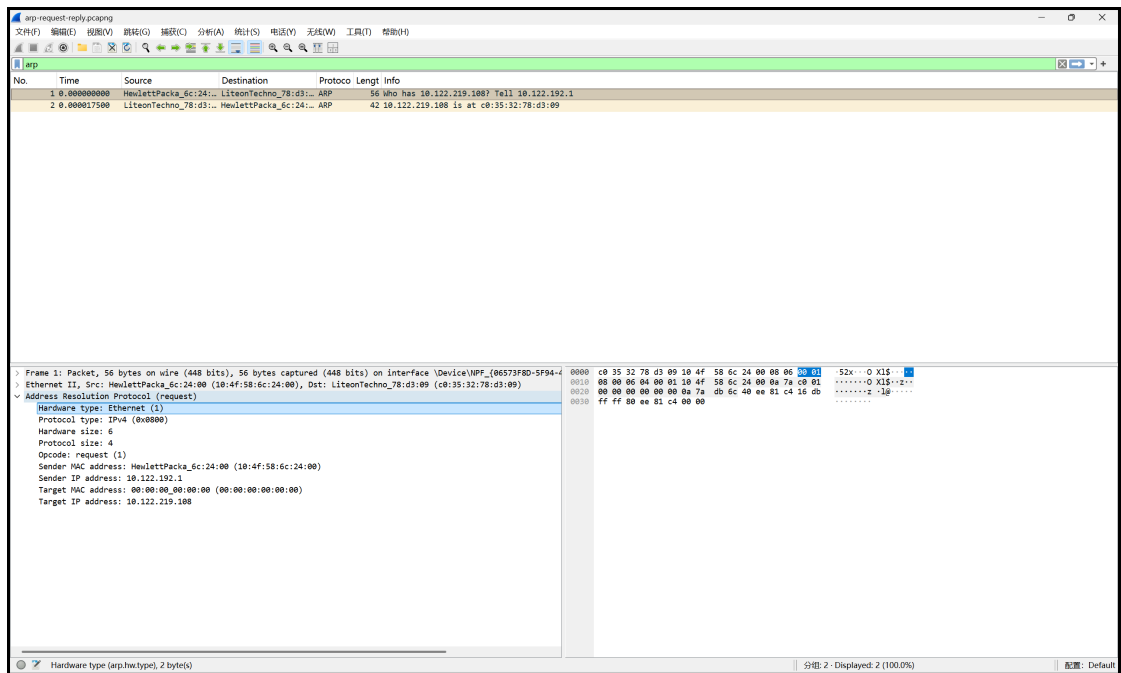


图 6 ARP Request / Reply Wireshark 截图（见图）

本节结论：ARP 通过 Opcode 1/2 完成「询问—应答」配对，将 IP 地址映射为 MAC 地址；本捕获中网关主动发起询问、本机单播回复，说明 ARP 既可用于广播发现，也可在已知 MAC 时采用单播询问。

TCP 协议分析

抓包文件：`captures/tcp/tcp-http.pcapng`（`curl -4 http://example.com, 10.122.219.108:60247 ⇄ 172.66.147.243:80`）。

TCP 首部字段

以帧 1（客户端 SYN，`60247 → 80`，`http://example.com`）为例：

字段	长度	捕获示例	功能
Source Port	16 bit	60247	源端口
Destination Port	16 bit	80	HTTP 端口
Sequence Number	32 bit	524890319	初始序号 ISN
Acknowledgment Number	32 bit	0	SYN 无确认号
Header Length	4 bit	32 bytes	含 MSS 选项
Flags	若干位	SYN	请求连接
Window Size	16 bit	65535	接收窗口
Urgent Pointer	16 bit	0	无紧急数据
Options (MSS)	可变	1460	最大段长度

表 10 TCP 首部字段（帧 1）

三次握手

连接 `10.122.219.108:60247 ⇄ 172.66.147.243:80`（帧 1 - 3）：

步骤	方向	Flags	Seq	Ack	说明
1	Client→Server	SYN	524890319	—	客户端请求连接
2	Server→Client	SYN, ACK	4130144482	524890320	Ack = Seq+1 ✓
3	Client→Server	ACK	524890320	4130144483	Ack = Seq+1 ✓

表 11 TCP 三次握手

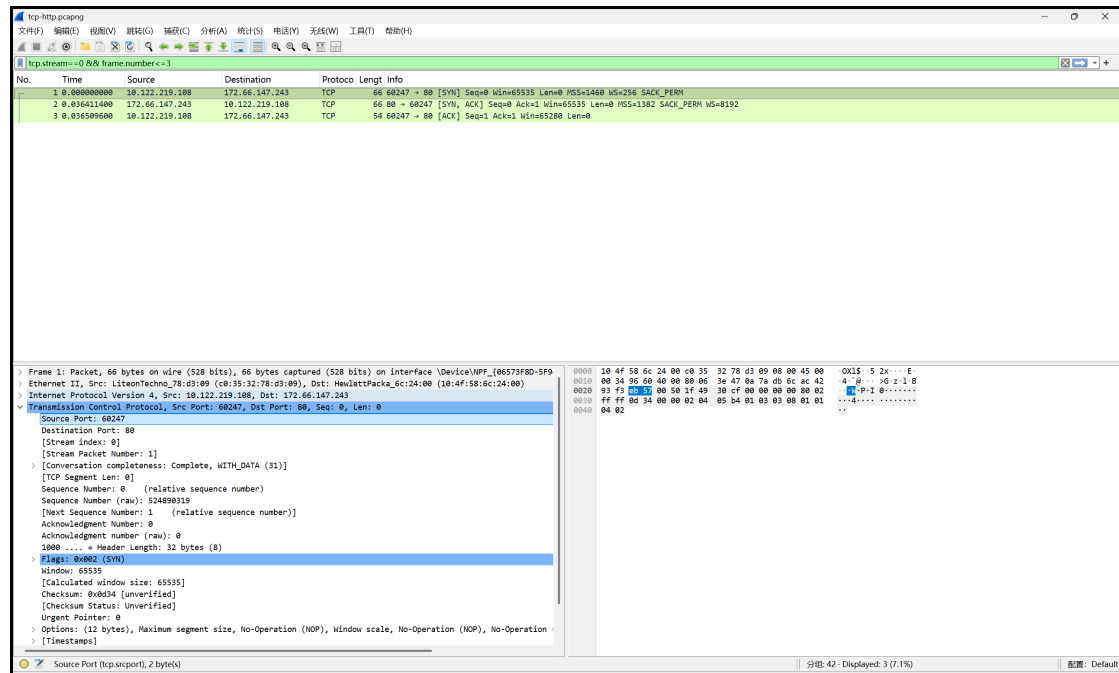


图 7 TCP 三次握手 Wireshark 截图

数据传输

帧 4 客户端发送 Seq=524890320, Len=75 ; 帧 5 服务器 Ack=524890395 。

验证: $524890395 = 524890320 + 75$ ✓

1. Window Size (帧 5): 65535 (16 bit 字段值), 表示接收方当前可接收的数据量。
2. MSS: 客户端 1460, 服务器 1382。

差错控制与窗口管理: 使用 tcp.analysis.retransmission 与 tcp.analysis.duplicate_ack

过滤检查后, 本次连接未发现重传或重复确认, 因此将其作为稳定 TCP 会话进行序号与确认号分析。本次 TCP 抓包中, 未观察到明显的 Retransmission、Duplicate ACK 或 RST 异常重置, 因此可以认为该次 HTTP 通信过程较为稳定, 没有出现需要重传恢复的丢包现象。TCP 的差错控制主要体现在确认应答、序号管理和校验机制上: 发送方每发送一段数据, 接收方会根据已经连续收到的字节数返回确认号; 若某段数据丢失, 接收方无法推进确认号, 发送方会在超时或收到重复 ACK 后重传相应数据。本次实验中帧 4 客户端发送 Seq=524890320, Len=75, 帧

5 服务器返回 `Ack=524890395`，正好满足 `524890395 = 524890320 + 75`，说明接收方已经正确收到这 75 字节数据。窗口字段则体现 TCP 的流量控制能力，接收方通过 Window Size 告知发送方当前可接收的数据量，避免发送方过快发送导致接收缓冲区溢出。

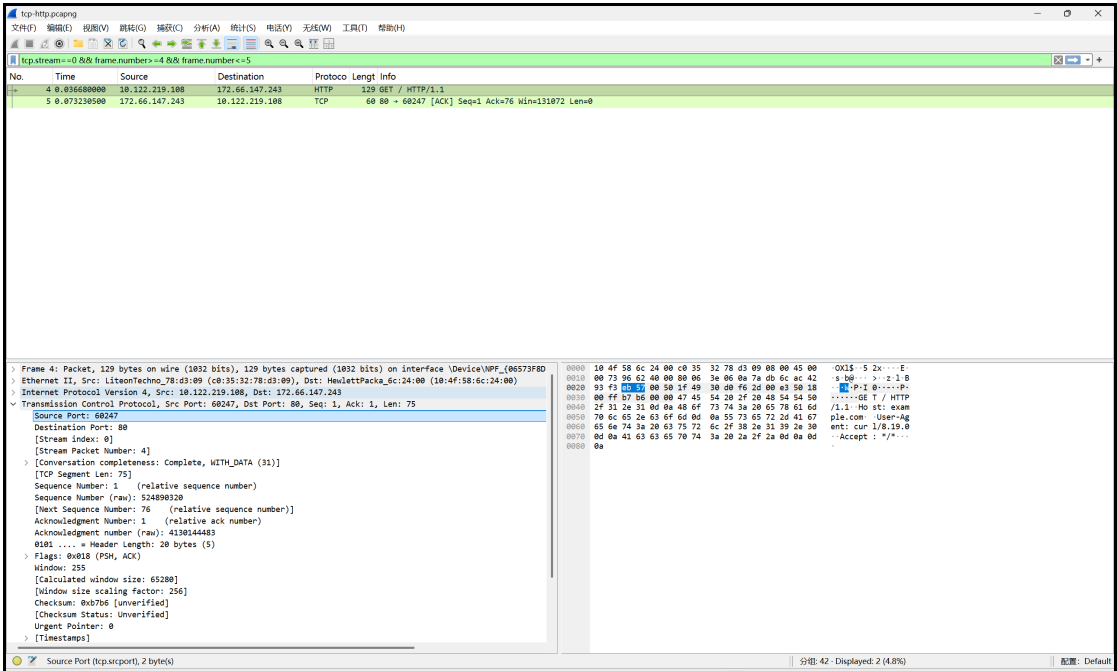


图 8 TCP 数据传输 Wireshark 截图

四次挥手

连接释放（帧 9 - 11，标准 FIN 关闭，无 RST）：

步骤	帧	方向	Flags	Seq	Ack	说明
1	9	Client→Server	FIN, ACK	524890395	4130145356	客户端请求关闭
2	10	Server→Client	ACK	—	524890396	确认 FIN， Ack=u+1 ✓
3	10	Server→Client	FIN	4130145356	524890396	服务器 FIN （同帧合并）
4	11	Client→Server	ACK	524890396	4130145357	最终确认， Ack=v+1 ✓

表 12 TCP 四次挥手（逻辑四步）

帧 10 将「对 FIN 的 ACK」与「服务器 FIN」合并发送，属常见优化；逻辑上仍对应四次挥手中间两步。

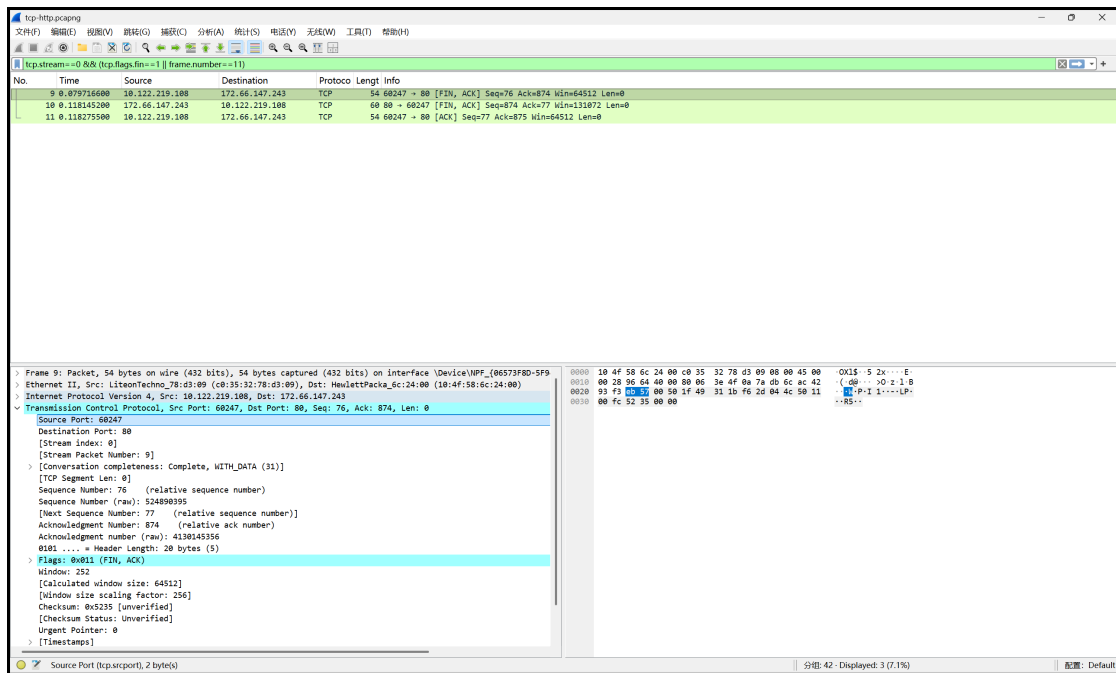


图 9 TCP 四次挥手 Wireshark 截图

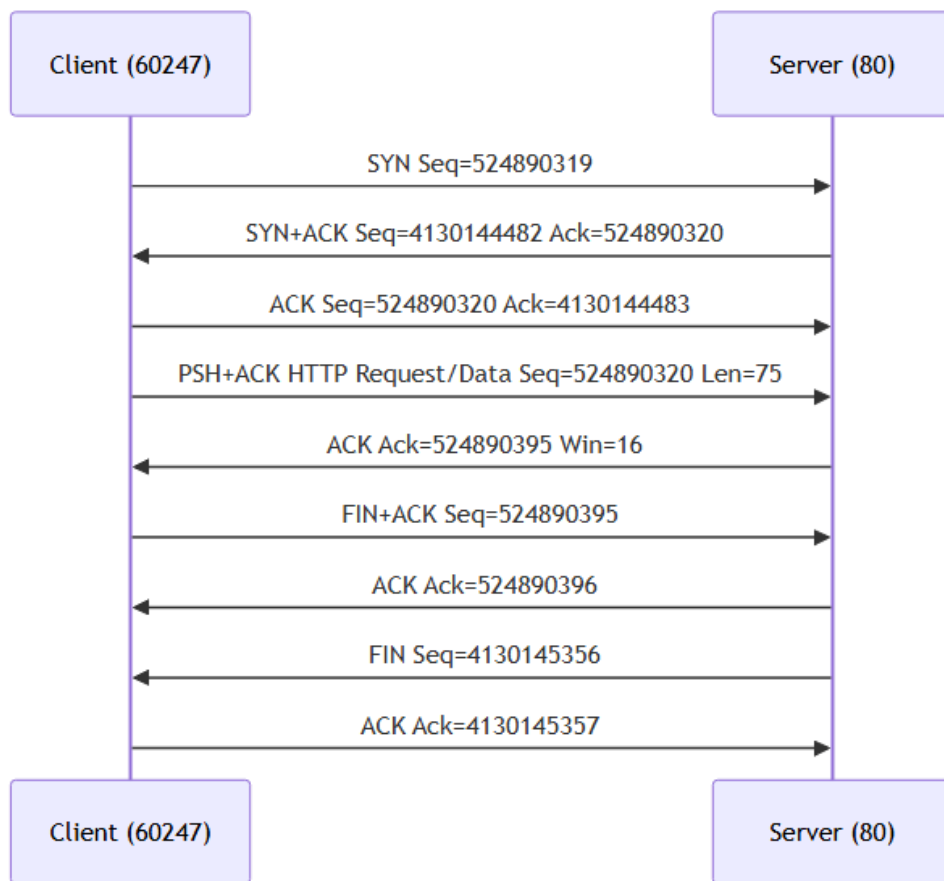


图 10 TCP 完整通信过程时序图（握手 → 数据传输 → 挥手）

本节结论：本次 HTTP 会话完整呈现了 TCP 连接建立（三次握手）、可靠数据传输（序号 + 确认号 + 窗口）与连接释放（四次挥手）三阶段；结合抓包可验证序号递增、确认累积与 FIN 关闭机制，符合指导书对 TCP 报文段格式及连接管理的要求。

实验实现与工作量说明

本次实验除完成 Wireshark 抓包与协议字段分析外，还进行了较完整的工程化整理工作。实验过程中，我没有只停留在截图层面，而是将抓包文件、字段摘录、校验脚本、时序图和最终报告源文件进行了统一组织，形成了可复现的实验材料。

数据采集方面，本实验分别针对 ICMP、DHCP、ARP、TCP 四类典型协议设计了独立的抓包流程。ICMP 部分既捕获了普通 `ping www.bupt.edu.cn` 报文，也额外构造了 `ping -l 8000` 的大包场景，用于观察 IP 分片过程；DHCP 部分通过 `ipconfig /release` 与 `ipconfig /renew` 人为触发地址释放和重新申请，从而获得完整的 DORA 流程；ARP 部分通过清空 ARP 缓存并访问网关触发地址解析；TCP 部分尝试过 HTTPS 流量，但由于部分连接会快速 RST 关闭，最终改用 `curl -4 http://example.com` 捕获更适合分析三次握手、数据传输和四次挥手的 HTTP 流量。所有抓包结果均按协议分类保存于 `captures/` 目录，便于后续复查和引用。

字段分析方面，我逐帧展开 Wireshark 的协议树，手工抄录了 IP、ICMP、DHCP、ARP、TCP 的关键字段，并将字段值整理为表格。对于 IP 协议，不仅记录了 Version、Header Length、Total Length、Identification、Flags、Fragment Offset、TTL、Protocol、Header Checksum 等字段，还进一步手工验证了 IPv4 首部校验和。对于 TCP 协议，不仅记录了端口、序号、确认号、标志位、窗口大小和 MSS，还结合真实抓包验证了 `Ack = Seq + 1`、`Ack = Seq + Len` 等序号变化规律。通过这种方式，报告中的结论不是直接引用教材，而是从实际抓包数据中推导得到。辅助验证方面，我编写了 IPv4 首部校验和验证脚本 `scripts/verify_ipv4_checksum.py`，用于对 Wireshark 中提取出的 20 字节 IP 首部进行交叉验证。脚本按照 IPv4 首部校验和算法，将校验和字段置零，按 16 bit 分组进行反码求和，并对最终结果取反，输出结果与 Wireshark 显示的 `0x6820` 一致。此外还编写了 `scripts/validate_phase6.py` 用于提交前检查抓包完整性、插图、占位符与 TCP 序号规律，以及 `scripts/capture_wireshark_screenshots.py` 用于自动截取 Wireshark 三栏真实界面图。这些脚本的作用是避免只凭肉眼抄录结论，同时也加深了对 IP 首部校验机制的理解，并保证报告插图可重复生成。

报告组织方面，我使用 Typst 编写实验报告源文件 `report/lab2-report.typ`，并将实验过程中的分析笔记（`notes/`）、抓包截图（`report/assets/`）和 Mermaid 时序图（`diagrams/`）整合到统一文档中。报告不是直接在 Word 中堆叠截图，而是按「实验目的一实验环境一实验步骤一协议分析一实现说明一问题解决一实验心得」的结构组织。对于 DHCP DORA、TCP 三次握手、

TCP 数据传输与 TCP 四次挥手等过程，我额外绘制了时序图，使协议交互过程更加直观。这样既保证了报告结构清晰，也方便后续修改、重新编译和复现实验结果。

从工作量上看，本次实验主要包括四部分：第一，完成多轮抓包与重抓，确保每类协议都有可分析的有效数据；第二，对多个 `.pcapng` 文件逐帧筛选和字段摘录，整理成结构化表格；第三，编写校验脚本、自动化截图脚本和时序图，对关键协议机制进行验证和可视化；第四，使用 Typst 完成实验报告排版和 PDF 编译。整体工作量不仅包括最终报告中的文字和截图，也包括背后的抓包文件整理、字段核对、脚本验证和文档工程化生成过程。

代码与文件规模说明

本次实验相关文件包括报告源码、协议分析笔记、校验脚本、时序图文件以及抓包数据文件。报告主文件为 `report/lab2-report.typ`，用于组织全文结构、表格、图片和说明文字；分析笔记位于 `notes/` 目录，用于记录各协议的字段提取过程；时序图位于 `diagrams/` 目录，用于描述 DHCP DORA、TCP 三次握手、数据传输和四次挥手流程；辅助脚本位于 `scripts/` 目录，其中 `verify_ipv4_checksum.py` 用于验证 IPv4 首部校验和，`validate_phase6.py` 用于最终提交检查，`capture_wireshark_screenshots.py` 用于批量生成 Wireshark 真截图。抓包数据统一保存在 `captures/` 目录下，并按 ICMP、DHCP、ARP、TCP 分类管理。

这些文件共同构成了本次实验的完整材料链路：先通过 Wireshark 生成 `.pcapng` 抓包文件，再从抓包文件中提取协议字段，随后使用脚本对关键字段进行验证，最后将分析结果、截图和时序图整合进 Typst 报告并编译为 PDF。相比只提交截图式实验报告，这种方式更便于复查字段来源，也能保证实验结论具有可验证性和可复现性。

下表为项目主要源码与文档的行数统计（统计日期：2026-06-19）：

类别	目录/文件	行数	作用
报告源码	report/lab2-report.typ	约 800	组织正文、表格、图片与附录
校验脚本	scripts/verify_ipv4_checksum.py	32	IPv4 首部校验和验证
截图脚本	scripts/capture_wireshark_screenshots.py	204	批量生成 Wireshark 三栏截图

类别	目录/文件	行数	作用
检查脚本	scripts/ validate_phase6.py	141	提交前六项自动检查
分析笔记	notes/（8 个 .md）	520	协议字段摘录、抓包步骤与过滤器
时序图	diagrams/（5 个 .mmd）	37	DHCP DORA 与 TCP 全流程图

表 13 实验工程文件规模统计

工程目录下 `report/`、`notes/`、`diagrams/`、`scripts/` 四类目录共 29 个文件；报告 Typst 源码与 Python 辅助脚本合计约 1200 行，分析笔记与时序图源码约 560 行。

实验中遇到的问题 and 解决方案

1. DHCP 先出现 Release 包

现象: `ipconfig /release` 产生 DHCP Release (帧 1, Transaction ID `0x9F16ECB3`), 与后续 DORA 的 Transaction ID 不同。

解决: 分析时按 Transaction ID `0x348015F3` 筛选帧 2 - 5, 忽略 Release 帧; 在报告中单独说明 Release 属于 `/release` 操作, 不属于 DORA 流程。

2. ARP 由网关主动询问

现象: Request 由网关 `10.122.192.1` 发出, 非本机广播询问网关; 以太网目的地址为本机 MAC 的单播 ARP。

解决: 仍为一组有效 Opcode 1/2 配对, 重点分析字段与单播回复机制, 并在报告中说明广播/单播差异。

3. TCP HTTPS 连接快速 RST 关闭

现象: 首次抓取 `tcp-https.pcapng` 时, HTTPS 连接常以 RST 快速关闭, 不符合标准四次挥手分析; 且首次 HTTP 抓包混有较多 IPv6 流量。

解决: 改用 `curl -4 http://example.com` 强制 IPv4, 保存 `tcp-http.pcapng`, 使用 `tcp.stream==0` 分析标准 FIN 挥手; 必要时重抓直至握手、传输、挥手帧完整。

4. Wireshark 截图需三栏同框

现象: 老师要求插图须含 Packet List、Packet Detail、Packet Bytes, 手工逐张截图耗时且易漏展协议层。

解决: 编写 `capture_wireshark_screenshots.py`, 按过滤器与目标帧自动打开 Wireshark、展开协议树并截图, 覆盖 `report/assets/` 下 8 张 PNG。

实验心得

通过本次实验，我从「只会看协议名称」逐步过渡到「能通过字段解释报文行为」：Wireshark 三栏让抽象首部变成可验证的数据；手工计算 IP 校验和（`0x6820`）加深了对反码求和的理解；IP 分片表直观展示了 MTU 限制；DHCP 四步广播/单播差异、ARP 地址解析、TCP 序号 +1 规律，都在抓包中得到印证。

本次实验不是简单截图交差，而是完成了「抓包 → 筛包 → 字段核对 → 手工计算 → 脚本验证 → 时序图绘制 → Typst 报告排版 → PDF 编译」的完整流程。多轮重抓（如 TCP 从 HTTPS 改为 HTTP、从混用 IPv6 改为 `curl -4`）让我认识到：协议分析的质量取决于数据是否干净、可解释；一份合格的实验报告，背后往往有多次抓包筛选与字段交叉核对。从截图式实验转向字段验证式实验，是我最大的收获之一。

在分析方法上，我学会了从「单个包」上升到「完整会话」：先用 `tcp.stream==0` 锁定一条 TCP 流，再依次观察握手、数据传输与挥手；DHCP 则按 Transaction ID 把 Release 与 DORA 区分开。遇到 DHCP Release 混入、ARP 由网关主动询问、HTTPS 快速 RST 等问题时，我没有简单丢弃抓包，而是通过过滤器、重抓和字段对照逐一解决——这种排查思路与今后网络故障定位是一致的。

在工具使用上，显示过滤器是分析效率的关键——先用 `icmp`、`dhcp`、`arp`、`tcp.stream==0` 等表达式缩小范围，再逐层展开协议树，比在海量无关流量中翻找高效得多。`tshark -T fields` 适合批量提取字段，`verify_ipv4_checksum.py` 适合验证手工计算，与 Wireshark GUI 形成互补。将 Mermaid 时序图渲染为 PNG 插入报告，也比在 PDF 中直接展示源码更符合「画出消息序列图」的要求。

今后排查网络问题，我会先用显示过滤器定位可疑流，再对照协议树与 Packet Bytes 核对首部字段；遇到校验和、序号等可计算量，优先用脚本交叉验证，避免抄录错误。把抓包、笔记、脚本、图表和报告源码放在同一仓库里管理，比零散保存截图和 Word 文档更利于复查与迭代——这也是我今后做课程实验和网络调试时愿意坚持的方式。

可复现性说明

复现实验结果的步骤如下：

1. 使用 Wireshark 打开 `captures/` 下对应 `.pcapng` 文件；
2. 应用报告中列出的 Display Filter（如 `icmp.type==8`、`bootp.id==0x348015f3`、`tcp.stream==0` 等）；
3. 根据帧号定位目标报文（如 IP 帧 1、DHCP 帧 2-5、TCP stream 0 帧 1-11）；
4. 在 Packet Detail 中逐层展开协议树，抄录字段并与报告表格对照；
5. 对 IP 校验和，使用 `scripts/verify_ipv4_checksum.py` 对 Packet Bytes 中的 20 字节首部复算；
6. 使用 `typst compile report/lab2-report.typ report/lab2-report.pdf` 重新编译报告。

由于实验环境为校园 Wi-Fi，网络中存在广播包、IPv6 流量、后台应用流量以及网关主动 ARP 等干扰，因此分析时需要通过过滤器和关键字段筛选目标报文。对于未在本次真实抓包中出现的 TCP 重传场景，报告只给出机制解释，不伪造重传报文。

附录：工程目录结构

本实验仓库按协议与职能分层组织，主要目录如下：

```
wireshark-protocol-analysis/
├─ captures/                # 抓包文件 (.pcapng)
│   ├─ icmp/                # icmp-normal、大包分片
│   ├─ dhcp/                # dhcp-dora
│   └─ arp/                 # arp-request-reply
│   └─ tcp/                 # tcp-http (HTTP 分析用)
├─ notes/                   # 协议分析笔记与过滤器参考
├─ diagrams/                # Mermaid 时序图源码
├─ scripts/
│   ├─ verify_ipv4_checksum.py # IPv4 首部校验和验证
│   ├─ validate_phase6.py     # 提交前六项检查
│   └─ capture_wireshark_screenshots.py # 批量截取 Wireshark 真图
├─ docs/                    # 实验要求与参考资料
└─ report/
    ├─ lab2-report.typ       # Typst 报告源码
    ├─ lab2-report.pdf       # 编译成品
    └─ assets/               # 报告插图 (Wireshark 截图)
```

图 2 实验工程目录结构

